

Render --backend

Databse – railway sql

Netlify – frontend

1st cretae db ----- 2nd work with frontend ----- 3rd work with backend

Railway MySQL – Step by Step (START HERE)

1  Go to:

 <https://railway.app>

2  Login with GitHub

3  Click:

STEP 1: Create FREE MySQL Database (Railway)

You are on **New project** screen.

 **DO THIS:**

1. Click **Database** (the option you highlighted)

STEP 1.1: Choose MySQL

2. Click **MySQL**

1.2: Database gets created automatically

Railway will now:

- Create MySQL instance
- Generate credentials
- Start it (takes ~10 seconds)

You will land on **MySQL service page**.

STEP 2: Copy Database Credentials

On the **MySQL** page:

Click variables tab

You will see something like:

Variable	Example
MYSQL_HOST	containers-us-west-xxx.railway.app
MYSQL_PORT	3306
MYSQL_DATABASE	railway
MYSQL_USER	root
MYSQL_PASSWORD	xxxxxx

☐ **DO NOT CHANGE ANYTHING**

Just **copy these values**.

Example ()

```
MYSQL_DATABASE
Railway

MYSQL_PUBLIC_URL
mysql://root:AmRDuXqLMdYtSbJEqjHihgUaXzdfhXII@switchback.proxy.rlwy
.net:51971/railway

MYSQL_ROOT_PASSWORD
AmRDuXqLMdYtSbJEqjHihgUaXzdfhXII

MYSQL_URL
mysql://root:AmRDuXqLMdYtSbJEqjHihgUaXzdfhXII@mysql.railway.internal:
3306/railway

MYSQLDATABASE
Railway

MYSQLHOST
mysql.railway.internal
```

```
MYSQLPASSWORD
AmRDuXqLMdYtSbJEqjHihgUaXzdfhXII
```

```
MYSQLPORT
3306
```

```
MYSQLUSER
root
```

in sts

update application.properties

with exact railway properties example :

Application Name

spring.application.name=Investment-Banking-Backend

Server Configuration

server.port=8080

Database Configuration (env vars with local defaults)

spring.datasource.url=jdbc:mysql://\${MYSQLHOST:localhost}:\${MYSQLPORT:3306}/\${MYSQLDATAB
ASE:backend}

spring.datasource.username=\${MYSQLUSER:root}

spring.datasource.password=\${MYSQLPASSWORD:200318}

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

JPA Configuration

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

spring.jpa.properties.hibernate.format_sql=true

JWT Configuration

jwt.secret=your-secret-key-change-in-production-1234567890

jwt.expiration=86400000

CORS Configuration

cors.allowed-origins=http://localhost:4200

Step 2: Set environment variables locally in STS

1. Go to **Run** → **Run Configurations** → **Spring Boot App** → **Environment**.
2. Add these variables for your local MySQL:

Name	Value
MYSQL_HOST	localhost
MYSQL_PORT	3306
MYSQL_DATABASE	backend
MYSQL_USER	root
MYSQL_PASSWORD	200318

STS will use these when you run the app locally.

Step 3: Test locally

- Run your Spring Boot app in STS.
- It should connect to your **local MySQL** using the environment variables (or the defaults in `application.properties`).

If still fails update application properties with this

```
# Application Name
spring.application.name=Investment-Banking-Backend

# Server Configuration
server.port=${PORT:8080}

# Database Configuration
spring.datasource.url=${MYSQL_PUBLIC_JDBC_URL:jdbc:mysql://localhost:3306/backend}
spring.datasource.username=${MYSQLUSER:root}
spring.datasource.password=${MYSQLPASSWORD:200318}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.properties.hibernate.format_sql=true
```

```
# JWT Configuration
jwt.secret=your-secret-key-change-in-production-1234567890
jwt.expiration=86400000

# CORS Configuration
cors.allowed-origins=http://localhost:4200
```

in render

environmental variables add this

MYSQL_PUBLIC_JDBC_URL=jdbc:mysql://switchback.proxy.rlwy.net:51971/railway?useSSL=false&allowPublicKeyRetrieval=true

MYSQLUSER=root

MYSQLPASSWORD=AmRDuXqLMdYtSbJEqjHihgUaXzdfhXll

Just start the service now booom !

Backend will works now

For frontend

Go to environment.prod.ts file

Their update link which u got a link from render

Example

```
export const environment = {
  production: true,
  apiUrl: 'https://investment-banking-backend.onrender.com'
};
```

In app module.ts

providers: [

 { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }

]

Example :

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';
import { ReactiveFormsModule, FormsModule } from '@angular/forms';

import { AppRoutingModuleModule } from './app-routing-module';
import { AppComponent } from './app';
import { SharedModule } from './modules/shared/shared.module';
import { AuthInterceptor } from './core/interceptors/auth.interceptor';
import { LoginComponent } from './modules/auth/login.component';
import { DealListComponent } from './modules/deals/deal-list/deal-list.component';
import { DealFormComponent } from './modules/deals/deal-form/deal-form.component';
import { DealDetailsComponent } from './modules/deals/deal-details/deal-details.component';
import { UserManagementComponent } from './modules/admin/user-management.component';
import { AddNoteDialogComponent } from './modules/deals/add-note-dialog/add-note-dialog.component';
import { ConfirmDialogComponent } from './modules/shared/components/confirm-dialog/confirm-dialog.component';
import { EmptyStateComponent } from './modules/shared/components/empty-state/empty-state.component';
import { LoadingSpinnerComponent } from './modules/shared/components/loading-spinner/loading-spinner.component';

@NgModule({
  declarations: [
    AppComponent,
    LoginComponent,
    DealListComponent,
    DealFormComponent,
    DealDetailsComponent,
    UserManagementComponent,
    AddNoteDialogComponent,
    ConfirmDialogComponent,
    EmptyStateComponent,
    LoadingSpinnerComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModuleModule,
```

```

    BrowserModule,
    HttpClientModule,
    SharedModule,
    ReactiveFormsModule,
    FormsModule
  ],
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }

```

Create `netlify.toml`

In your **Angular project root** (same level as `package.json`):

```

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200

```

□ This tells Netlify:

“Send all routes to Angular”

next

ng build --configuration production

This creates:

`dist/<project-name>/`

example

: G:\cap\Frontend\Investment-Banking-Frontend\dist\Investment-Banking-Frontend

Deploy to Netlify (FREE)

Option A (Easiest – Drag & Drop)

1. Go to □ <https://app.netlify.com>
2. Login with GitHub
3. Click “Add new site → Deploy manually”

4. Drag:
5. Add this in build commands
6. npm install && npx ng build --configuration production

dist/<project-name>

in backend

On Controller Class

Example:

```
@CrossOrigin(origins = "**")
@RestController
@RequestMapping("/api/deals")
public class DealController {
    ...
}
Do in every controller class cross origin things

If security config their then add this
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();

    configuration.setAllowedOrigins(Arrays.asList(
        "http://localhost:4200",
        "https://your-site-name.netlify.app"  -----this one -----
    ));

    configuration.setAllowedMethods(Arrays.asList(
        "GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"
    ));

    configuration.setAllowedHeaders(Arrays.asList(
        "Authorization", "Content-Type", "Accept"
    ));

    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new
    UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
}
```

Again push every thing into github
Now deploy into netlify frontend comes live

Then come to backend

☐ DONE!

